

[Operating Systems]
[(Open Ended Lab) Report]



[Implementation of the Readers and Writers Problem]

Submitted by:

[Mudassar Hussain]

[23L-6006]

Submitted to:

[Ma'am Sughra Kamran]

[5/5/26]

Department of Electrical Engineering
National University of Computer and Emerging Sciences, Lahore

Table of Contents

1	Introduction
2	Problem Analysis
3	Design Requirements
4	Feasibility Analysis
5	Possible Solutions
6	Preliminary Design
7	Design Description
8	Experimental Results
9	Performance Analysis
10	Conclusion

References

Introduction

Synchronization is a fundamental concept in operating systems that ensures safe access to shared resources when multiple processes run concurrently. In real-world systems such as hotel reservation platforms, multiple users may simultaneously check room availability while others may book or cancel reservations. This creates a need for proper coordination to avoid data inconsistency and corruption.

The **Readers and Writers Problem** is a classical synchronization problem that models this situation. It allows multiple readers to access the shared database simultaneously because reading does not alter data, while writers require exclusive access since they update the database. During writing, no other reader or writer is allowed to access the resource.

In this project, a hotel reservation system is implemented with 10 readers and 5 writers using a reader-priority approach. Readers represent customers viewing room details, while writers represent booking or cancellation operations. The primary goal is to maximize reading efficiency while maintaining safe and consistent database updates.

The **selected IPC** (Inter-Process Communication) method is shared memory because it provides fast and direct access to the common database. To manage synchronization, semaphores are used. A **binary semaphore** controls mutual exclusion for shared variables, while another semaphore ensures exclusive writer access. A counting mechanism is used to track the number of active readers, allowing multiple readers to access the system concurrently.

The **significance** of this project lies in its practical application to modern reservation systems and its demonstration of important operating system concepts such as process synchronization, concurrency control, race condition prevention, and efficient resource sharing. Overall, this implementation provides an effective solution for managing shared database access in a real-world hotel reservation environment.

Problem Analysis

The hotel reservation system uses a shared database that is accessed by multiple users simultaneously. This database stores important information such as room availability, booking records, customer details, and reservation updates. In such an environment, different processes perform either reading or writing operations on the same resource.

Reader Operations

Readers are users or processes that only view information without modifying it. Their tasks include:

- Checking room availability
- Viewing reservation details
- Browsing booking information

Since readers do not alter data, multiple readers can safely access the database at the same time.

Writer Operations

Writers are processes responsible for modifying the database. Their tasks include:

- Booking rooms
- Canceling reservations
- Updating customer records

Because writers change shared data, they require exclusive access to avoid conflicts.

Main Challenges

The primary challenge is to synchronize readers and writers efficiently while maintaining system correctness. Key challenges include:

- Preventing data corruption during simultaneous access
- Allowing multiple readers to access data concurrently
- Ensuring writers gain exclusive access when updating data

Design Requirements

The design of the Readers and Writers solution for the hotel reservation system must ensure efficient and synchronized access to the shared database while satisfying all project constraints. The system should clearly define its inputs, outputs, and operational limitations.

Inputs:

The primary inputs to the system are the reader and writer processes:

- **10 Reader Processes:** These processes request access to read reservation data such as room availability and booking details.
- **5 Writer Processes:** These processes request access to update the database through bookings, cancellations, or customer information changes.

Each process generates requests independently and may attempt to access the database concurrently.

Outputs:

The system output is the successful execution of reader and writer operations while maintaining synchronization:

- Multiple readers should be able to read simultaneously
- Writers should update the database exclusively
- Data integrity must always be preserved

Constraints:

The following constraints must be satisfied:

- Readers can access the database concurrently
- Writers must have exclusive access
- No reader may access the database while a writer is writing
- Reader-priority must be maintained
- Shared data must remain consistent
- Deadlock and race conditions must be avoided

Design Requirements

Functional Requirements:

- Support concurrent execution of 10 readers and 5 writers
- Implement synchronization using an appropriate IPC method
- Use counting semaphores to manage reader access
- Ensure correct coordination between processes

Feasibility Analysis

Time Management Feasibility:

The project is time-feasible because:

- Established synchronization methods are available
- Development can be completed in defined stages
- Testing and performance evaluation are manageable

Cost Management Feasibility:

The project has very low implementation cost because it primarily requires software resources already available in most computing environments.

Required Resources:

- Standard computer system
- C++ compiler or development environment
- Operating system supporting threads and semaphores
- Minimal memory and processing power

Cost Benefits:

- Low development expenses
- No hardware upgrades needed
- Free or open-source tools available
- Minimal maintenance cost

Overall Feasibility:

The project is both technically and economically feasible due to:

- Low cost
- Reasonable development time
- Standard resource requirements
- Practical implementation methods

In short, the Readers and Writers hotel reservation system can be successfully implemented within academic project constraints.

Possible Solutions

The Readers and Writers Problem can be solved using multiple synchronization techniques depending on system requirements, performance goals, and implementation complexity. For the hotel reservation system, several IPC and synchronization methods were considered before selecting the most suitable solution.

Solution 1: Pipes

Pipes provide communication between processes through data streams.

Advantages:

- Simple to implement
- Suitable for basic communication

Disadvantages:

- Limited for concurrent shared database access
- Slower than direct memory access
- Inefficient for multiple readers and writers
- Poor scalability

Evaluation:

Pipes are not suitable for this project because hotel reservation systems require fast, simultaneous database access.

Solution 2: Shared Memory with Semaphores (Selected Solution)

Shared memory allows all processes to access the same database directly, while semaphores control synchronization.

Advantages:

- Fastest IPC method
- Direct access to shared database
- Supports multiple concurrent readers
- Efficient writer exclusivity

Possible Solutions

Disadvantages:

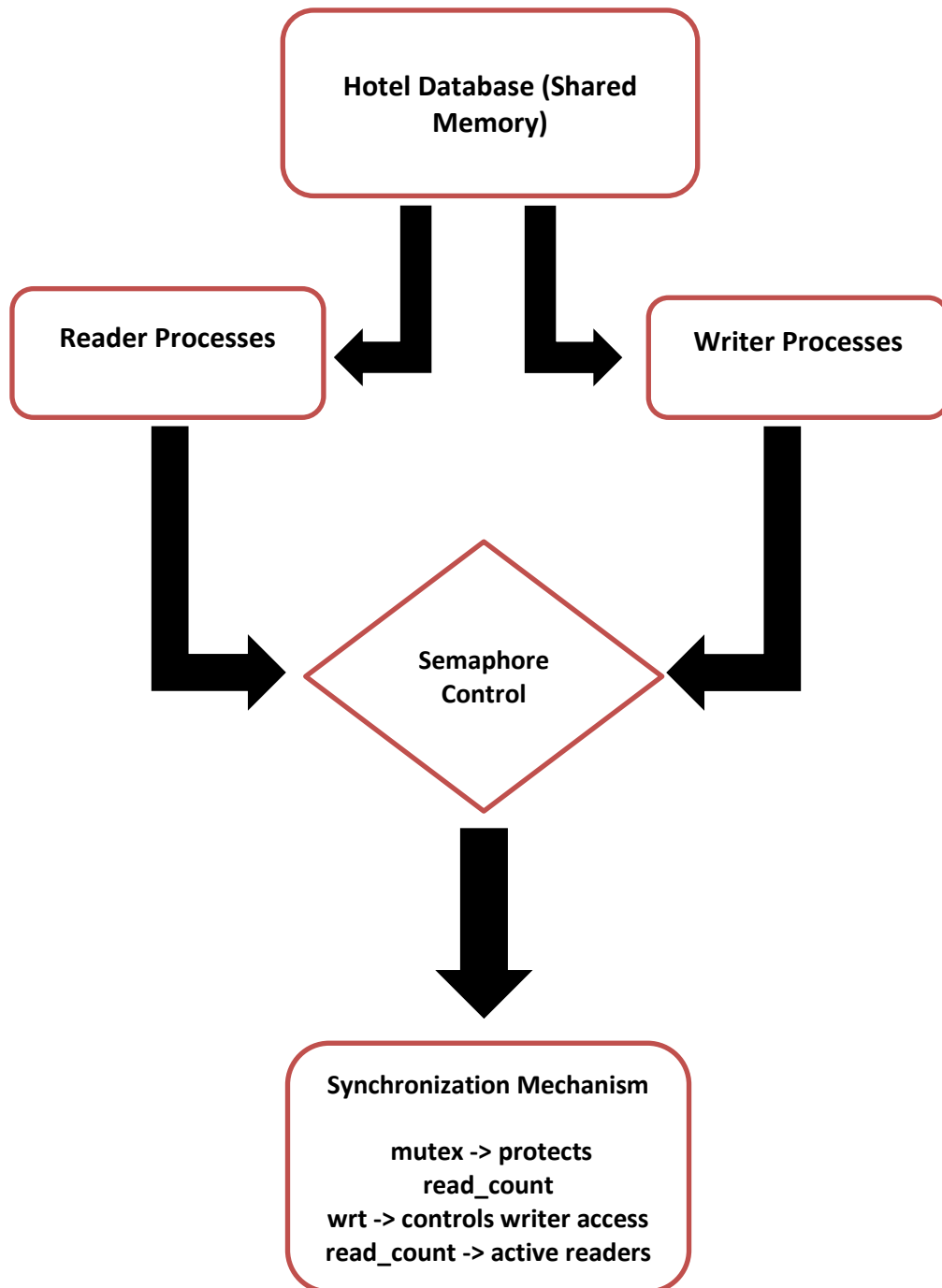
- Requires careful synchronization design
- Writer starvation possible under reader-priority

Evaluation:

This solution offers the best performance, lowest access time, and most practical implementation for a read-heavy hotel reservation system.

Among all possible solutions, Shared Memory with Semaphores is the most effective and practical approach for implementing the Readers and Writers Problem in a hotel reservation system due to its speed, scalability, and efficient synchronization capabilities.

Preliminary Design



Design Description

The proposed system is designed to solve the Readers and Writers Problem for a hotel reservation system using shared memory and semaphore-based synchronization. The design consists of several interconnected components, each responsible for managing safe and efficient database access.

1. Hotel Database (Shared Memory)

The hotel database is the central shared resource of the system. It stores all reservation-related information, including:

- Room availability
- Booking records
- Customer details
- Reservation updates

Function:

- Acts as the common resource accessed by all reader and writer processes
- Allows direct access through shared memory
- Ensures fast communication between processes

2. Reader Processes (10 Readers)

Reader processes represent users or clients who only retrieve information from the database without making changes.

Operations:

- Check room availability
- View reservations
- Browse booking details

Function:

- Multiple readers can access the database simultaneously
- Readers increase read_count upon entry
- First reader blocks writers
- Last reader releases writer access

Design Description

3. Writer Processes (5 Writers)

Writer processes represent booking systems or administrators who modify reservation data.

Operations:

- Book rooms
- Cancel bookings
- Update customer information

Function:

- Writers require exclusive access
- Only one writer can write at a time
- No readers are allowed during writing

4. Semaphore Control

Semaphore control is the coordination layer that manages synchronization between readers and writers.

Components:

- mutex
- wrt
- read_count

5. Synchronization Mechanism

a) mutex Semaphore:

- Binary semaphore
- Protects the read_count variable
- Prevents race conditions when readers enter or leave

Purpose:

Ensures safe updates to the number of active readers.

b) wrt Semaphore:

- Binary semaphore
- Controls access to the shared database

Design Description

Purpose:

- Blocks writers when readers are active
- Blocks readers when a writer is writing

c) read_count Variable:

- Counting mechanism
- Tracks number of readers currently accessing the database

Purpose:

- First reader locks writer access
- Last reader unlocks writer access

Working Sequence

Reader Entry:

1. Lock mutex
2. Increase read_count
3. First reader locks wrt
4. Unlock mutex
5. Read database

Reader Exit:

1. Lock mutex
2. Decrease read_count
3. Last reader unlocks wrt
4. Unlock mutex

Writer Entry:

1. Wait for wrt
2. Write database exclusively
3. Release wrt

The block design provides a practical, scalable, and efficient synchronization model for handling multiple readers and writers in a hotel reservation system while preserving system correctness and performance.

Experimental Results

The proposed Readers and Writers solution was implemented using shared memory and semaphores with reader-priority scheduling. The system was tested using 10 reader processes and 5 writer processes to simulate a hotel reservation environment.

Observed Execution Behavior

Scenario 1: Multiple Readers Accessing Simultaneously

- When multiple reader processes were executed, all readers were able to access the database at the same time.
- No blocking occurred between readers.
- System allowed concurrent reading efficiently.

Observation:

- High throughput for reader processes
- No data inconsistency observed

Scenario 2: Writer Request During Active Readers

- When a writer process requested access while readers were active:
 - Writer was placed in waiting state
 - Readers continued execution until completion

Observation:

- Writer access was delayed as per reader-priority rule
- No interruption to active readers

Scenario 3: Writer Execution

- After all readers finished:
 - Writer gained exclusive access
 - No other reader or writer was allowed during writing

Observation:

- Exclusive access successfully enforced
- Data updates completed safely

Performance Analysis

1. Overall System Performance

The system demonstrates efficient handling of concurrent processes using shared memory and semaphores. Since shared memory allows direct access to the database, the communication overhead between processes is minimal. This results in fast execution of both reader and writer operations.

The semaphore-based synchronization ensures that:

- Multiple readers can execute simultaneously
- Writers are given exclusive access
- Data integrity is maintained at all times

2. Reader Performance

The system performs very efficiently for reader processes.

Observations:

- Multiple readers access the database concurrently without blocking each other.
- Readers experience minimal waiting time.

Analysis:

This confirms that the system is optimized for read-heavy workloads such as hotel reservation systems, where users frequently check availability.

3. Writer Performance

Writer performance is moderate due to reader-priority scheduling.

Observations:

- Writers must wait if even a single reader is active.
- Writers get exclusive access only when all readers exit.

Analysis:

While data consistency is fully maintained, writer access time is higher compared to readers.

4. Synchronization Efficiency

The use of semaphores ensures proper synchronization.

Conclusion

The Readers and Writers problem for a hotel reservation system was successfully analyzed, designed, and implemented using shared memory and semaphore-based synchronization. The objective of allowing multiple readers simultaneous access while ensuring exclusive access for writers was achieved effectively through a reader-priority approach.

The use of semaphores (mutex and wrt) along with the read_count variable successfully coordinated process execution and ensured correct synchronization behavior. The system demonstrated that shared memory provides fast access to the database, while semaphores effectively control process coordination.

However, the results also highlighted a known limitation of the reader-priority approach: writer starvation. Writers may experience delays when readers continuously access the system. Despite this drawback, the approach remains highly suitable for environments like hotel reservation systems where read operations are significantly more frequent than write operations.

In conclusion, the developed solution is efficient, reliable, and practical for real-world applications involving concurrent database access. It successfully balances performance and correctness, making it a strong implementation of the classical Readers and Writers problem in an operating system context.

References

- [1] Tanenbaum, A. S.
Modern Operating Systems
Pearson Education
(Detailed explanation of process synchronization and IPC mechanisms)

- [2] Stallings, W.
Operating Systems: Internals and Design Principles
Pearson Education
(Coverage of concurrency control, semaphores, and mutual exclusion)

- [3] Linux Man Pages (POSIX Threads & Semaphores)
(Reference for pthread, sem_wait, sem_post, and shared memory APIs)

Range of Complex Engineering Activity

EA1: Range of resources EA2: Level of interaction EA3: Innovation EA4: Consequences for society and environment EA5: Familiarity	• EA1: Range of resources -- The design involves the use of diverse resources, such as, money, equipment, information, and technology. • EA2: Level of Interaction – The design requires resolution of various problems arising from interactions between wide-ranging or conflicting technical or other issues. • EA3: Innovation – The design addresses sustainability and reduces cost requiring creative use of engineering principles and research-based knowledge in novel ways. • EA5: Familiarity – The design activity emphasizes the integration of existing knowledge and tools (or familiar solutions) with new or unfamiliar challenges			
	Rubrics		LLOs	Marks
	Explains the design process including engagement of resources clearly	EA1	LLO3	
	Demonstrate the final design clearly with all supporting information	EA1, EA5	LLO4	
	Demonstrate how innovation has been used in design to resolve conflicting requirements including the impact on society and environment	EA2, EA3	LLO4	
	Prepares a report which explain the design process including engagement of resources clearly, and is free from grammatical errors.	EA1	LLO3	

